

# Optimization of a GEMM Implementation using Intel AMX

Yusuke Endo  
Kyushu university  
Fukuoka, Japan  
endo.yusuke.491@s.kyushu-u.ac.jp

Satoshi Ohshima  
Research Institute for Information  
Technology, Kyushu University  
Fukuoka, Japan  
ohshima@cc.kyushu-u.ac.jp

Takeshi Nanri  
Research Institute for Information  
Technology, Kyushu University  
Fukuoka, Japan  
nanri.takeshi.995@m.kyushu-u.ac.jp

## Abstract

In high-performance computing, general matrix multiplication (xGEMM) routines form the core of Level-3 BLAS kernels, which enables efficient matrix operations. Among these, the low-precision GEMM such as BFloat16 has become indispensable in machine learning and deep learning because it reduces memory usage and power consumption. To meet this demand, recent hardware platforms are equipped with dedicated matrix computation units separate from the CPU, and research efforts have focused on maximizing their performance. Intel's Advanced Matrix Extension (AMX) is one such hardware accelerator designed specifically for low-precision matrix operations. In this study, we implement and optimize matrix multiplication-accumulation using AMX by applying blocking and tile register level optimizations and evaluate its performance. Our results demonstrate a performance improvement in the range of 7.27–20.40% compared to that of the BFloat16 GEMM implementations provided by MKL and OpenBLAS.

## CCS Concepts

- **Computing methodologies** → **Vector / streaming algorithms**;
- **Mathematics of computing** → *Mathematical software performance*.

## Keywords

Intel AMX, GEMM, OpenBLAS, MKL

### ACM Reference Format:

Yusuke Endo, Satoshi Ohshima, and Takeshi Nanri. 2026. Optimization of a GEMM Implementation using Intel AMX. In *SCA/HPCAsia 2026: Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia 2026)*, January 26–29, 2026, Osaka, Japan. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3773656.3773660>

## 1 Introduction

The growing scale of modern deep learning models has significantly increased their computational costs and memory consumption. Consequently, training with conventional single-precision floating-point (FP32) arithmetic has become a significant bottleneck in terms of performance and energy efficiency. To address this issue, the brain floating point 16 (BFloat16) format has gained prominence because it balances numerical stability and efficiency by featuring the same exponent range as FP32 while being represented with a

16-bit width, similar to that of FP16. Currently, BFloat16 is widely used to train large-scale language models and Transformer-based architectures, establishing it as the de facto standard for precision in modern deep learning. Therefore, optimizing algorithms and implementations based on BFloat16 is paramount for future research and applications in machine learning [5].

General matrix multiplication (GEMM) is a core computational kernel in machine learning and deep learning. Many primary numerical operations in the forward and backward passes of a model can be reduced to matrix multiplications, which account for the majority of the computational cost. Consequently, significant optimization efforts in both software and hardware have focused on GEMM. Modern processors are often equipped with specialized low-precision matrix multiplication units such as Google's Tensor Processing Unit (TPU) [4] and NVIDIA's Tensor Cores. Extensive research is being conducted to maximize the performance of these units [7][6]. Intel Advanced Matrix Extensions (AMX) [3] is one such low-precision matrix multiplication unit, featured in the 4th Generation Intel Xeon Scalable processors.

Research on matrix multiplication with precision correction using these low-precision units has been advancing. Accelerating large-scale, low-precision matrix computations is an essential and critical challenge for applications in scientific computing.

Numerical linear algebra libraries such as Intel MKL and OpenBLAS provide GEMM function implementations for various precisions. Among these, the GEMM function for the BFloat16 format includes implementations that leverage AMX. In this study, we optimize matrix multiplication-addition operations using AMX, drawing inspiration from the implementation methodology in OpenBLAS, and we compare its performance against existing libraries. Our implementation achieved a speedup of 7.27–20.40% compared to that of the BFloat16 GEMM implementations provided by MKL and OpenBLAS.

The remainder of this paper is organized as follows. Section 2 reviews the related work. Section 3 introduces the overview and operations of AMX. Section 4 describes the implementation of matrix multiplication-addition using AMX in the OpenBLAS numerical linear algebra library. In Section 5, we propose a new implementation method for matrix multiplication-addition using AMX. Section 6 presents experiments and performance evaluations of the proposed method. Finally, Section 7 provides conclusions and discusses future work.

## 2 Related Work

Several studies have explored the implementation of matrix multiplication using specialized hardware units. NVIDIA's Tensor Cores have shown to significantly accelerate GEMM, not only for machine learning but also for high-performance computing (HPC)



This work is licensed under a Creative Commons Attribution 4.0 International License. *SCA/HPCAsia 2026, Osaka, Japan*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2067-3/26/01

<https://doi.org/10.1145/3773656.3773660>

applications. For example, Paul et al. [12] designed a general tensor–tensor multiplication framework, and Hao et al. [13] optimized GEMM computations for tall-and-skinny matrices by leveraging these specialized cores.

A primary challenge in utilizing such accelerators for HPC is the precision requirement because many scientific applications demand single or double precision to maintain numerical accuracy. To address this, researchers actively developed mixed-precision methods that can harness the speed of low-precision hardware while preserving high-precision results.

One prominent mixed-precision method is the Ozaki scheme, which computes high-precision matrix multiplication using lower-precision arithmetic units. Mukunoki et al. utilized this scheme to implement double-precision GEMM (DGEMM) on FP16 tensor cores, achieving higher accuracy and performance compared to those of the standard DGEMM implementations [8]. Similarly, Ootomo et al. demonstrated that they could compute DGEMM faster than both cuBLAS and the existing FP16-based methods by using integer matrix multiplication units with the Ozaki scheme, thereby successfully accelerating a quantum circuit simulation while maintaining FP64 accuracy [9]. Complementing these algorithmic approaches, Valero-Lara et al. explored new frameworks using NVIDIA’s TF32 and by proposing a novel TF64 framework to simplify the execution of high-precision arithmetic on low-precision hardware [14].

Beyond achieving higher precision, other research focused on maximizing the performance of tensor cores. Yan et al. conducted a detailed analysis of the NVIDIA Turing architecture, revealing that memory bandwidth is the new bottleneck for half-precision GEMM (HGEMM). Based on their findings, they developed an optimized routine that significantly outperformed NVIDIA’s cuBLAS library [15]. Abdelfattah et al. achieved substantial speedups over cuBLAS by optimizing batched HGEMM for very small matrices, effectively using low-level warp matrix functions [1].

The application of these low-precision, high-throughput engines is not limited to matrix multiplication. Zhang et al. investigated their use for accelerating fundamental matrix computations such as QR factorization [16]. They showed that although significant speedups are possible, achieving robust accuracy on neural engines requires co-designing algorithms (e.g., using recursive Gram–Schmidt) and applications, which often requires incorporating techniques such as iterative refinement to avoid numerical issues.

The OpenBLAS library, which is the focus of our study, also includes a matrix multiply-accumulate implementation using AMX, which enables mixed-precision GEMM [10]. In this research, we aim to improve and accelerate this mixed-precision GEMM implementation within OpenBLAS, building on insights gained from the aforementioned studies.

### 3 Matrix Operations with Intel AMX

#### 3.1 Overview of Intel AMX

The Intel AMX is a hardware accelerator specialized for matrix numerical operations, introduced in the 4th Generation Intel Xeon Scalable processors and later. It supports matrix operations for BFloat16 and int8 data types. AMX includes eight 2D registers called “tiles,” each with a maximum size of 1 KB. Matrix operations

on the data within these tiles are executed by an accelerator engine attached to the tiles, which is referred to as the tile matrix multiply (TMUL) unit.

#### 3.2 Intel AMX Operations

The operations for matrix calculations in AMX are shown in Listing 1. For BFloat16 data, a matrix of up to  $16 \times 32$  elements is loaded into a tile. The `_tile_dpbfp16ps` instruction then computes the dot product of the BFloat16 floating-point pairs, returning the result as a matrix of up to  $16 \times 16$  single-precision (float32) elements [2].

The fundamental computation performed by AMX is a dot product; there is no single instruction for a complete matrix multiplication. Therefore, reordering the matrix data and applying the dot product operation sequentially to obtain the final matrix product is necessary to perform a full matrix multiplication using AMX. The specific procedure is described in Section 4.

### 4 OpenBLAS Implementation

OpenBLAS is a numerical linear algebra library that provides the implementations of the basic linear algebra subprogram (BLAS) and the linear algebra package (LAPACK). Unlike to similar libraries such as MKL whose implementation details are not public, OpenBLAS is open-source, allowing the examination of its specific implementation methods. OpenBLAS includes a half-precision GEMM function that utilizes AMX. This function performs a mixed-precision computation where the input matrices A and B are in the BFloat16 format, and the output matrix C is accumulated and stored in single-precision.

#### 4.1 Cache Blocking

The blocking strategy for the matrix multiply-accumulate operation  $C = C + A \times B$  using AMX in OpenBLAS within a single-threaded environment is shown in Figure 1. The input matrices A, B, and C are all stored in the row-major format.

Given matrix dimensions  $C : M \times N$ ,  $A : M \times K$ , and  $B : K \times N$ ; OpenBLAS performs cache blocking as follows:

- Matrix A: Partitioned along the column direction into blocks of size  $k = 1024$ .
- Matrix B: Partitioned along the row direction into blocks of size  $k = 1024$  and along the column direction into blocks of size  $n = 256$ .

Furthermore, buffers are allocated to store the blocked matrices: *BUFFER\_A* with a size of  $M \times k$  and *BUFFER\_B* with a size of  $n \times k$ . Subsequently, the corresponding sub-blocks of A and B are packed into these buffers.

As shown in Figure 2, during packing, *BUFFER\_A* is rearranged into a  $16 \times 32$  row-block layout, which means that elements are stored contiguously in  $16 \times 32$  blocks, and once a block is filled, the next block is stored by prioritizing the row direction. Meanwhile, *BUFFER\_B* is reordered into a  $32 \times 16$  transposed block layout. This arrangement ensures that applying the AMX dot-product operation yields the correct matrix multiplication result (Figure 3).

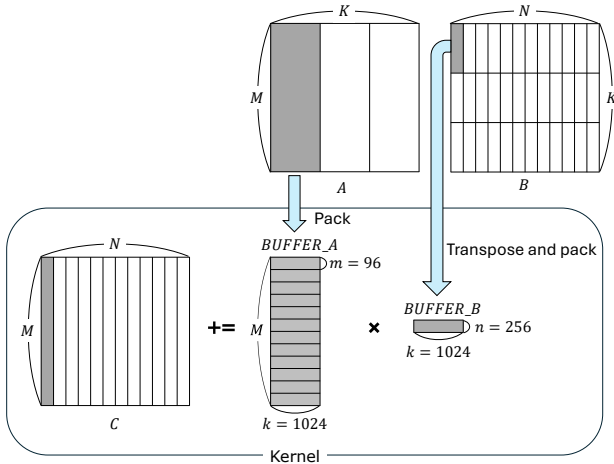
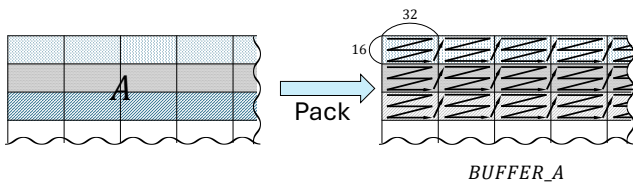
The usage of *BUFFER\_A* is conditional. When its blocks are computed with *BUFFER\_B* for the first time—that is, when the corresponding column indices of C are in the range of 0–255—*BUFFER\_A* is further partitioned by rows into blocks of size  $m = 96$ .

**Listing 1: Operation of the `_tile_dpbfp16ps` instruction**

```

1 void _tile_dpbfp16ps(__tile dst, __tile a, __tile b)
2 FOR m := 0 TO dst.rows - 1
3   tmp := dst.row[m]
4   FOR k := 0 TO (a.colsb / 4) - 1
5     FOR n := 0 TO (dst.colsb / 4) - 1
6       tmp.fp32[n] += FP32(a.row[m].bf16[2*k+0]) * FP32(b.row[k].bf16[2*n+0])
7       tmp.fp32[n] += FP32(a.row[m].bf16[2*k+1]) * FP32(b.row[k].bf16[2*n+1])
8     ENDFOR
9   ENDFOR
10  write_row_and_zero(dst, m, tmp, dst.colsb)
11 ENDFOR
12 zero_upper_rows(dst, dst.rows)
13 zero_tileconfig_start()

```

**Figure 1: Blocking strategy in OpenBLAS****Figure 2: Packing layout for BUFFER\_A**

In all other cases, the operation is performed directly between the full *BUFFER\_A* and *BUFFER\_B*.

Figure 4 shows the blocking strategy for a multi-threaded environment. Matrix C is equally partitioned so that each sub-block of C is nearly square. We use a four-thread case as an example. Although the cache block sizes are the same as that in the single-threaded case, the difference is that a sub-block of size  $M/4 \times k$  from matrix A is copied into each thread's *BUFFER\_A*.

For a square matrix, matrix C is divided into a  $2 \times 2$  grid of four blocks, and each thread is assigned one block to compute. Each

thread has its own independent *BUFFER\_A* and *BUFFER\_B*. Initially, each thread performs computations using its kernel function on the buffers it has packed.

Subsequently, threads responsible for the same block of rows in matrix C exchange their *BUFFER\_A*s. Then, they perform another round of computation with the received buffer and their own *BUFFER\_B*.

## 4.2 Tile Operations

Figure 5 shows the tile register operations used in the OpenBLAS kernel function on *BUFFER\_A* and *BUFFER\_B*.

- The operation processes two rows from matrix A and two columns from matrix B at a time.
- One tile register is allocated for each of these vectors (four tiles in total:  $A_{2m,k}$ ,  $A_{2m+1,k}$ ,  $B_{k,2n}$ , and  $B_{k,2n+1}$ ).
- The remaining four registers are allocated for the result sub-matrix of C ( $C_{2m,2n}$ ,  $C_{2m,2n+1}$ ,  $C_{2m+1,2n}$ , and  $C_{2m+1,2n+1}$ ).
- Elements from C, *BUFFER\_A*, and *BUFFER\_B* are loaded into their respective tile registers.
- Then, the multiply-accumulate operations are performed as:

$$\begin{aligned}
 C_{2m,2n} &+= A_{2m,k} \times B_{k,2n}, \\
 C_{2m,2n+1} &+= A_{2m,k} \times B_{k,2n+1}, \\
 C_{2m+1,2n} &+= A_{2m+1,k} \times B_{k,2n}, \\
 C_{2m+1,2n+1} &+= A_{2m+1,k} \times B_{k,2n+1}.
 \end{aligned}$$

In this paper, we refer to this sequence of tile register operations as the **Base-kernel**. This method is considered optimal because it minimizes the number of load operations relative to the number of computations, utilizes all eight tile registers efficiently, and requires minimal processing overhead.

## 5 Proposed Optimizations

### 5.1 Optimization of Cache Blocking Size

In the existing OpenBLAS cache blocking strategy for the 4th Generation Xeon Scalable processors, the block size for matrix B (0.5 MB) is relatively small compared to the L2 cache size (2 MB). If the block size is too small relative to the cache size, it increases the number of accesses to the result matrix C, which can degrade the performance. Therefore, we anticipate that the performance can be

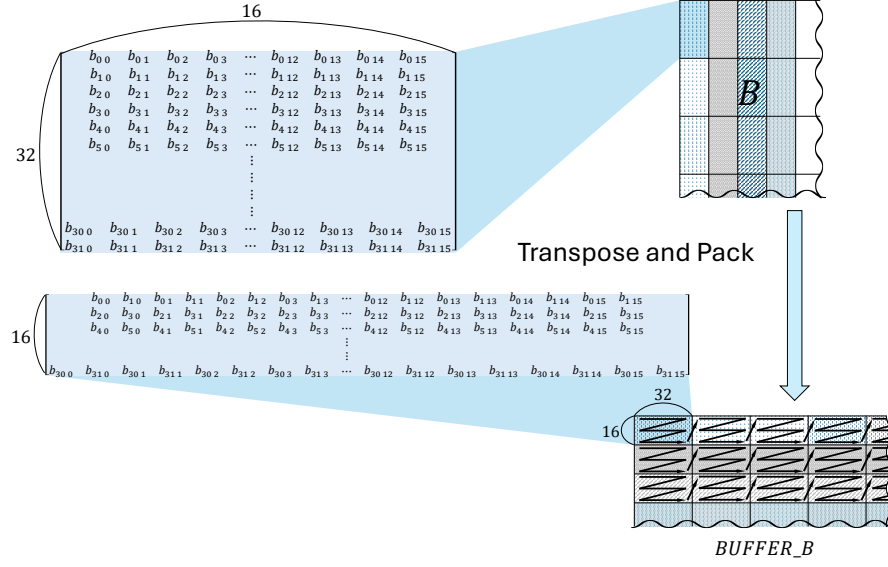


Figure 3: Transposing and packing for BUFFER\_B

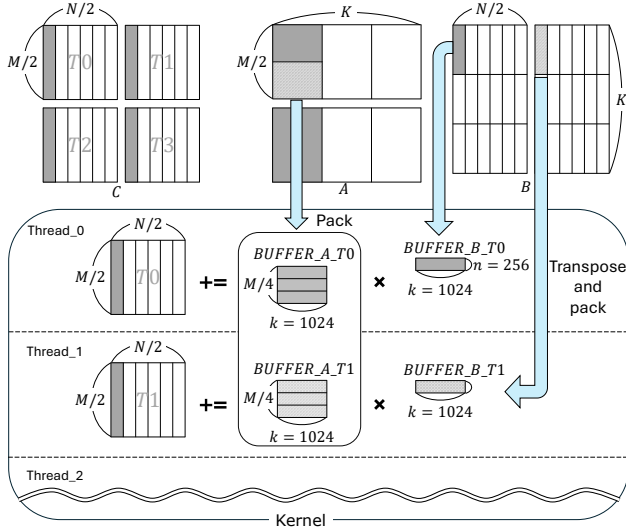


Figure 4: Blocking strategy for parallel execution (multi-threading) in OpenBLAS

improved by setting the block size to be as large as possible within the limits of the cache capacity.

## 5.2 Proposal of New Tile Operations

We propose new tile operation methods, as illustrated in Figures 6 and 7. The key feature of these methods for the matrix multiplication  $C = AB$  is treating one matrix (either  $A$  or  $B$ ) as a set of  $16 \times 3$  tiles and the other as a set of  $16 \times 2$  tiles. A total of six tile registers are allocated for the corresponding region of the result matrix  $C$ ,

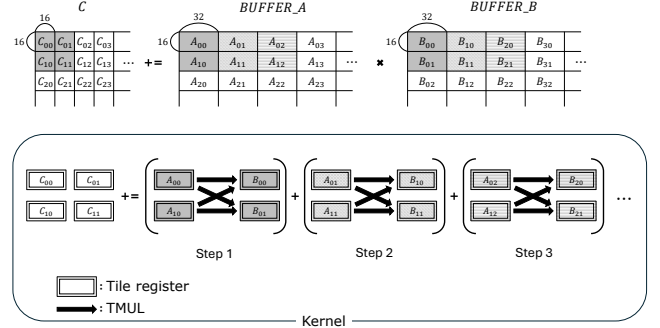


Figure 5: Tile operations in OpenBLAS (Base-kernel)

whereas the tiles from matrices  $A$  and  $B$  are loaded using the remaining two registers. Hereafter, we refer to the method that treats matrix  $A$  as  $16 \times 3$  tiles (Figure 6) as **Tiling\_A**, and the method that treats matrix  $B$  as  $16 \times 3$  tiles (Figure 7) as **Tiling\_B**.

**5.2.1 Operational Procedure for Tiling\_A.** The computational procedure for Tiling\_A is outlined in Figure 8. The numbers in the top-left corners of the blocks indicate the operational order. First, the target block of matrix  $C$  ( $C_{2m,2n}$ ,  $C_{2m,2n+1}$ ,  $C_{2m+1,2n}$ ,  $C_{2m+1,2n+1}$ ,  $C_{2m+2,2n}$ ,  $C_{2m+2,2n+1}$ ) is loaded into tile registers  $R_0$  through  $R_5$ . Then, the tiles from matrices  $A$  and  $B$  are loaded into registers  $R_6$  and  $R_7$  to perform the computations. The specific steps are as follows:

- (1) Load  $A_{2m,k}$  into  $R_6$  and  $B_{k,2n}$  into  $R_7$ . Add the multiply-accumulate result to  $R_0$  (corresponding to  $C_{2m,2n}$ ).
- (2) Load  $B_{k,2n+1}$  into  $R_7$ . Add the multiply-accumulate result to  $R_1$  (corresponding to  $C_{2m,2n+1}$ ).

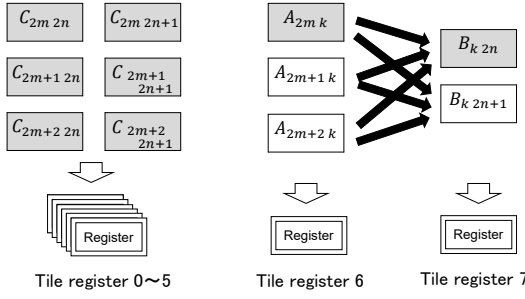


Figure 6: Proposed tile operation method: Tiling\_A

- (3) Load  $A_{2m+1,k}$  into  $R_6$ . Add the multiply-accumulate result to  $R_3$  (corresponding to  $C_{2m+1,2n+1}$ ).
- (4) Reload  $B_{k,2n}$  into  $R_7$ . Add the multiply-accumulate result to  $R_2$  (corresponding to  $C_{2m+1,2n}$ ).
- (5) Load  $A_{2m+2,k}$  into  $R_6$ . Add the multiply-accumulate result to  $R_4$  (corresponding to  $C_{2m+2,2n}$ ).
- (6) Reload  $B_{k,2n+1}$  into  $R_7$ . Add the multiply-accumulate result to  $R_5$  (corresponding to  $C_{2m+2,2n+1}$ ).

**5.2.2 Operational Procedure for Tiling\_B.** The computational procedure for Tiling\_B is shown in Figure 9. Similar to Tiling\_A, the target block of matrix  $C$  ( $C_{2m,2n}, \dots, C_{2m+1,2n+2}$ ) is loaded into  $R_0$  through  $R_5$ , and the tiles from  $A$  and  $B$  are loaded into  $R_6$  and  $R_7$ . The specific steps are:

- (1) Load  $A_{2m,k}$  into  $R_6$  and  $B_{k,2n}$  into  $R_7$ . Add the multiply-accumulate result to  $R_0$  (corresponding to  $C_{2m,2n}$ ).
- (2) Load  $B_{k,2n+1}$  into  $R_7$ . Add the multiply-accumulate result to  $R_1$  (corresponding to  $C_{2m,2n+1}$ ).
- (3) Load  $B_{k,2n+2}$  into  $R_7$ . Add the multiply-accumulate result to  $R_2$  (corresponding to  $C_{2m,2n+2}$ ).
- (4) Load  $A_{2m+1,k}$  into  $R_6$ . Add the multiply-accumulate result to  $R_5$  (corresponding to  $C_{2m+1,2n+2}$ ).
- (5) Reload  $B_{k,2n}$  into  $R_7$ . Add the multiply-accumulate result to  $R_3$  (corresponding to  $C_{2m+1,2n}$ ).
- (6) Reload  $B_{k,2n+1}$  into  $R_7$ . Add the multiply-accumulate result to  $R_4$  (corresponding to  $C_{2m+1,2n+1}$ ).

**5.2.3 Discussion on the Efficiency of the Proposed Methods.** In the proposed methods, seven load instructions are issued for six multiply-accumulate operations, which may seem inefficient at first glance because the number of loads exceeds the number of computations. However, two of the seven loads are reloads of recently used data, which makes them highly likely to result in L1 cache hits. Compared to the existing OpenBLAS method, where all loads are assumed to be L2 cache hits, our proposed methods have the potential to achieve higher performance based on the latencies of the cache hierarchy.

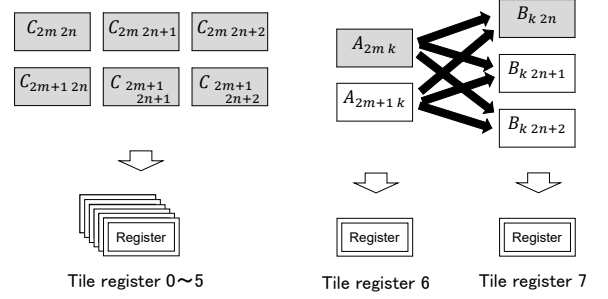


Figure 7: Proposed tile operation method: Tiling\_B

### 5.3 Optimizing Memory Access by Introducing Prefetch Instructions

The existing kernel function in OpenBLAS does not implement prefetch instructions to hide memory access latency. Focusing on this aspect, we aimed to improve performance by introducing prefetch instructions into the kernel function.

Within the three nested loops of the kernel, we prefetched data from *BUFFER\_A* into the L1 cache in the outermost  $m$ -loop. The objective is reducing the latency associated with data access by loading data, which is presumed to reside in the slower L3 cache or main memory, into the L1 cache in advance.

We did not apply prefetching to *BUFFER\_B* because its data are expected to reside in the faster L2 cache. For L2-to-L1 prefetching, we confirmed that the overhead from issuing the prefetch instructions actually outweighed the latency reduction benefits, resulting in overall performance degradation.

## 6 Performance Evaluation

### 6.1 Experimental Setup

We evaluated the performance of the matrix multiply-accumulate operation under the following settings to validate the effectiveness of the optimization methods proposed in this study.

**Evaluation Metric:** The processing time of GEMM is measured after initializing the elements of matrices  $A$ ,  $B$ , and  $C$  with random numbers.

**Target Operation:** The operation to be evaluated is  $C \leftarrow C + A \times B$ . Here,  $A$ ,  $B$ , and  $C$  are all real-valued square matrices stored in the row-major order. The computation is performed with no transpose operations.

**Matrix Sizes:** Although small matrices are prevalent in deep learning, this study focuses on large-scale computations crucial to scientific computing and HPC applications. This research primarily focuses on optimizing the kernel, which accounts for the majority of the computation time. Therefore, we target relatively large matrix sizes where the kernel processing becomes the bottleneck to clearly measure the effects of our optimizations. Specifically, measurements were conducted for the following 16 sizes:

- Dimensions as powers of two: 1024, 2048, 4096, 8192, and 16384.

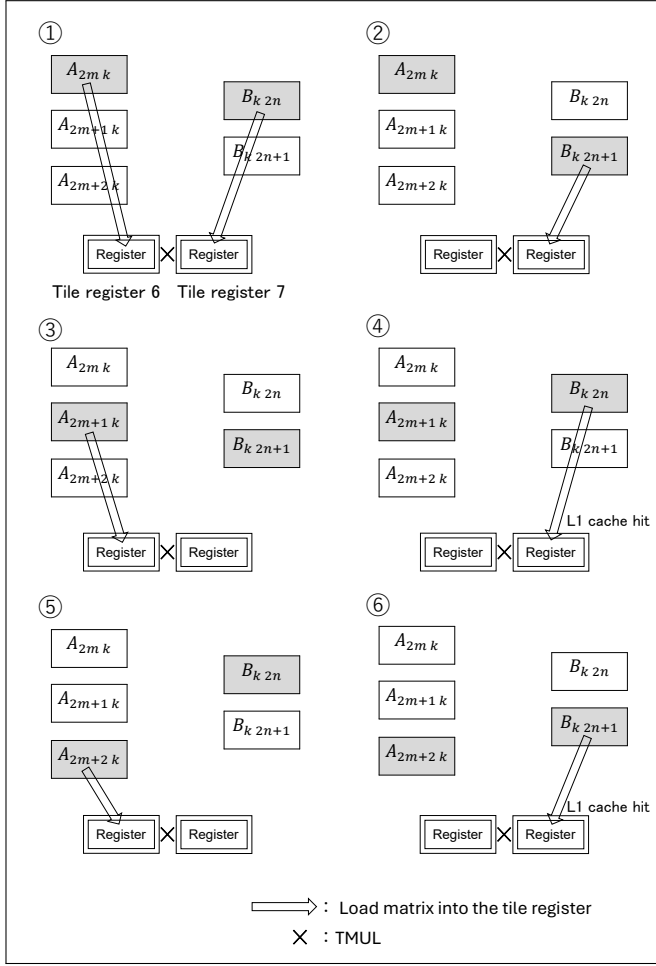


Figure 8: Computational order for Tiling\_A

- Dimensions in increments of 1000: from 10000 to 20000 in steps of 1000.

**Implementation Assumptions:** It is assumed that the AVX-512 instruction set is utilized for pre-processing steps such as data packing before the main computation.

## 6.2 Comparison Targets

For comparison, we used two GEMM functions: `cblas_gemm_bf16` from the Intel MKL library and `cblas_sbgemm` from OpenBLAS. As shown in Figure 10, both functions, similar to our implementation, perform mixed-precision matrix operations that produce a float32 matrix from two bf16 matrices.

## 6.3 Experimental Environment

The experiments were conducted on an Intel Xeon Platinum 8490H processor, which is part of Node Group A of the Genkai supercomputer at the Research Institute for Information Technology, Kyushu University. The CPU specifications for Genkai's Node Group A are listed in Table 1.

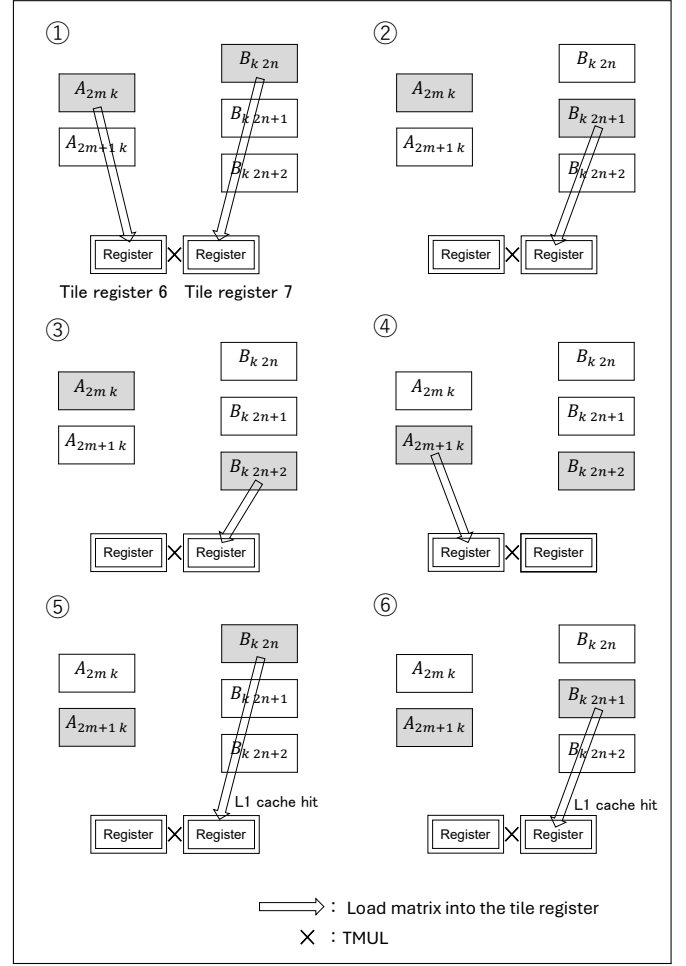


Figure 9: Computational order for Tiling\_B

Table 1: CPU Specifications for Genkai Node Group A

|                          |                                             |
|--------------------------|---------------------------------------------|
| Name                     | Intel Xeon Platinum 8490H (Sapphire Rapids) |
| Processor base frequency | 1.90 GHz                                    |
| Max turbo frequency      | 3.50 GHz                                    |
| Total cores              | 60                                          |
| Total threads            | 120                                         |
| L1 instruction cache     | 48 KB per core                              |
| L1 data cache            | 32 KB per core                              |
| L2 cache (Mid-level)     | 2 MB private per core                       |
| L3 cache (Last level)    | 1.875 MB per core                           |
| Memory types             | DDR5 4800 MHz                               |

Each CPU comprises four NUMA nodes, with 15 cores per node. To evaluate the performance within a single NUMA domain, we implemented both single-threaded and multi-threaded versions of our GEMM and benchmarked them with one, four, and eight threads. A single physical node was reserved for the experiments,

```

void cblas_gemm_bf16bf16f32(
    const CBLAS_LAYOUT Layout, const CBLAS_TRANSPOSE transA, const CBLAS_TRANSPOSE transB,
    const MKL_INT M, const MKL_INT N, const MKL_INT K, const float alpha,
    const MKL_BF16 *A, const MKL_INT lda, const MKL_BF16 *B, const MKL_INT ldb,
    const float beta, float *C, const MKL_INT ldc
);

void cblas_sbgemm(
    CBLAS_ORDER layout, CBLAS_TRANSPOSE transA, CBLAS_TRANSPOSE transB, blasint M, blasint N,
    blasint K, float alpha, const bfloat16 *A, blasint lda, const bfloat16 *B, blasint ldb,
    float beta, float *C, blasint ldc
);

```

Figure 10: Signatures of the GEMM functions used for comparison.

Table 2: Software Environment

|                  | MKL                                                                                                     | OpenBLAS, Ours                |
|------------------|---------------------------------------------------------------------------------------------------------|-------------------------------|
| OS               | Red Hat Enterprise Linux 8.8 (Ootpa)                                                                    |                               |
| Compiler         | gcc 13.3.1                                                                                              | gcc 13.3.1                    |
| Library version  | Intel oneAPI Math Kernel Library (oneMKL) 2025.1.3                                                      | OpenBLAS 0.3.29               |
| Compiler options | -O3 -march=native<br>-lmkl_intel_lp64<br>-lmkl_core<br>-lmkl_intel_thread<br>-liomp5 -lpthread -lm -ldl | -O3 -march=native<br>-fopenmp |

and the `numactl` command was used to bind the execution to a single NUMA node, matching the number of cores to the thread count. The software environment is detailed in Table 2. In the table, our implemented GEMM is referred to as “Ours.”

## 6.4 Experiment 1: Evaluating the Impact of Block Size on Performance

In this experiment, we evaluate the effect of the cache blocking parameters, block size  $k$  and  $n$ , on the performance of the GEMM operation.

**6.4.1 Methodology.** With the tile operation method fixed to **Base-kernel**, we measured the execution time while varying the block sizes across the following 15 combinations:

- Block size  $n$ : 256, 288, 480, 512, and 576 (five variations)
- Block size  $k$ : 1024, 1536, and 2048 (three variations)

For each condition, the same operation was executed 20 times, and the maximum observed FLOPS value across all problem sizes was adopted as the performance metric.

**6.4.2 Results and Discussion.** The performance of the existing libraries (OpenBLAS, MKL) is presented in Table 3, and the measurement results of our implementation are listed in Tables 4–6. In these tables, values that surpass the performance of MKL are highlighted with a gray background. Furthermore, graphs plotting the performance for the best-performing block size combination

Table 3: Performance of OpenBLAS and MKL (GFLOPS)

| Library  | One thread | Four threads | Eight threads |
|----------|------------|--------------|---------------|
| MKL      | 1163.74    | 4304.76      | 7968.88       |
| OpenBLAS | 1054.39    | 3927.94      | 7116.79       |

(indicated in bold in the tables) at each thread count are shown in Figures 11–13.

*Effect of Introducing Prefetching.* First, we compare our proposed method with the original OpenBLAS. The results for our method using the same block size as OpenBLAS ( $n = 256, k = 1024$ ) closely matched or exceeded OpenBLAS’s performance across all thread counts. The only implementation difference between the two is the presence of prefetch instructions, and therefore, this confirms that prefetching introduced in our study is effective for performance improvement.

*Impact of Varying Block Sizes.* For matrix sizes of 4096 and larger, performance improvements over the baseline were observed for many block size combinations. This can be attributed to an improved data reuse rate from setting the block sizes appropriately, leading to performance gains of up to 5.16% compared to that of MKL and up to 16.79% compared to that of the original OpenBLAS. However, a trend of performance degradation was observed when the block size ( $k \times n$ ) significantly exceeded the 2 MB L2 cache capacity. We presume that this is caused by an increased penalty from cache misses.

Conversely, the existing libraries were faster for smaller matrix sizes. With smaller problems, the data being operated on is more likely to already reside within the L2 cache. In such cases, the latency-hiding effect of prefetching is minimal, and the overhead of issuing the instructions itself likely had a detrimental effect on performance.

## 6.5 Experiment 2: Performance Evaluation of Different Tile Operations

In this experiment, we evaluate the performance effect of the two tile operation methods proposed in this study, `Tiling_A` and `Tiling_B`, by comparing their performance across various block sizes.



**Table 4: Experiment 1: Performance of Base-kernel Method with One Thread (GFLOPS)**

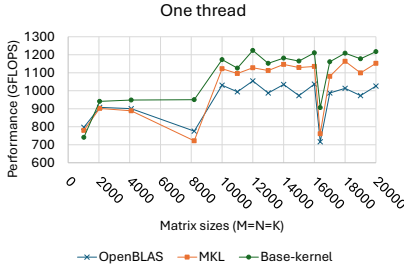
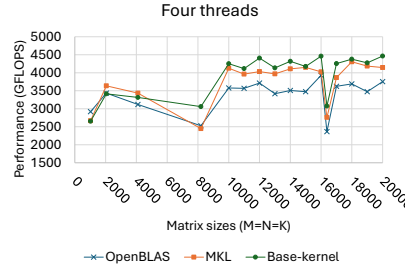
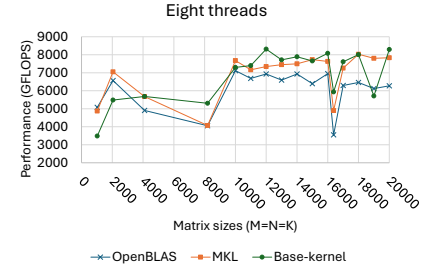
| Base-kernel one thread |          |                |          |
|------------------------|----------|----------------|----------|
| Block size             | k = 1024 | k = 1536       | k = 2048 |
| n = 256                | 1054.21  | 1123.42        | 1153.82  |
| n = 288                | 1074.08  | 1143.15        | 1173.03  |
| n = 480                | 1156.34  | <b>1223.80</b> | 814.26   |
| n = 512                | 1166.75  | 969.36         | 775.10   |
| n = 576                | 1176.93  | 1120.16        | 701.12   |

**Table 5: Experiment 1: Performance of Base-kernel Method with Four Threads (GFLOPS)**

| Base-kernel four threads |                |          |          |
|--------------------------|----------------|----------|----------|
| Block size               | k = 1024       | k = 1536 | k = 2048 |
| n = 256                  | 4011.76        | 4285.30  | 4389.40  |
| n = 288                  | 4078.87        | 4383.86  | 4424.80  |
| n = 480                  | 4381.21        | 4349.02  | 3199.87  |
| n = 512                  | 4410.70        | 3971.90  | 2927.71  |
| n = 576                  | <b>4464.41</b> | 3691.48  | 2732.22  |

**Table 6: Experiment 1: Performance of Base-kernel Method with Eight Threads (GFLOPS)**

| Base-kernel eight threads |          |                |          |
|---------------------------|----------|----------------|----------|
| Block size                | k = 1024 | k = 1536       | k = 2048 |
| n = 256                   | 7437.24  | 8053.12        | 7949.99  |
| n = 288                   | 7519.25  | 8181.42        | 8153.94  |
| n = 480                   | 7910.27  | <b>8312.22</b> | 6216.47  |
| n = 512                   | 8071.12  | 8244.16        | 5812.43  |
| n = 576                   | 8102.77  | 7007.71        | 5244.01  |

**Figure 11: Experiment 1: Best performance comparison with one thread.****Figure 12: Experiment 1: Best performance comparison with four threads.****Figure 13: Experiment 1: Best performance comparison with eight threads.**

**6.5.1 Methodology.** The experimental conditions were identical to those in Experiment 1, including the 15 block size combinations and the performance metric (the peak FLOPS value across all problem sizes, determined from the maximum of 20 runs for each size).

**6.5.2 Results and Discussion.** The measurement results for the Tiling\_A operation are presented in Tables 7–9, and the results for Tiling\_B are presented in Tables 10–12. In each table, the gray background indicates that the performance surpassed that of MKL. In addition, graphs plotting the highest performance achieved for each tile operation (indicated in bold in the tables) are shown in Figures 14–16.

The analysis of the results reveals different performance trends compared to the **Base-kernel** tile operation from Experiment 1, even when using the same block sizes.

**Evaluation of Tiling\_A.** Compared to the Base-kernel operation, no significant performance improvement was observed during the single-threaded execution. However, a trend of improved performance was confirmed in the multi-threaded executions.

**Evaluation of Tiling\_B.** Tiling\_B demonstrated consistent performance improvements over the Base-kernel operation across all single-threaded and multi-threaded execution conditions.

**Overall Evaluation.** Based on a comprehensive evaluation of the results from Experiments 1 and 2, the combination using **Tiling\_B** as the tile operation with a block size of ( $n = 480, k = 1536$ ) achieved the highest performance within the scope of our evaluation, demonstrating an improvement of up to 8.67% compared to that of MKL and up to 20.40% compared to that of the original OpenBLAS.

## 6.6 L1 Cache Misses for Each Implementation

We measured the number of L1 data cache misses for the OpenBLAS, Base-kernel, Tiling\_A, and Tiling\_B implementations using the block size configuration that yielded the shortest execution time for each. The measurements were conducted using “perf” [11], which is a performance analysis tool for Linux. The results are presented in Tables 17–19.

The miss counts were measured for the entire program, including all 20 GEMM operations, and it should therefore be considered indicative. However, the results confirm that our proposed tile operations (Tiling\_A and Tiling\_B) incur fewer L1 cache misses than the original tile operation used in OpenBLAS, thereby contributing to their higher performance. Notably, a comparison based on the total number of cache accesses or the resulting cache hit rate was not performed because perf cannot track cache accesses originating directly from AMX.

However, based on the algorithm, the total number of memory loads should follow the pattern: OpenBLAS/Base-kernel < Tiling\_A/Tiling\_B. Given that our measured cache misses show the opposite trend (OpenBLAS/Base-kernel > Tiling\_A/Tiling\_B), it is reasonable to infer that the Tiling\_A and Tiling\_B methods achieve a higher overall cache hit rate.

## 6.7 Comparison with Theoretical Peak Performance

We converted the performance of our best-performing configurations for the Base-kernel, Tiling\_A, and Tiling\_B implementations to GFLOPS and compared them against the theoretical peak performance of the hardware. The results are presented in Table 13.



**Table 7: Experiment 2: Performance of Tiling\_A with One Thread (GFLOPS)**

| Tiling_A one thread |          |                |          |
|---------------------|----------|----------------|----------|
| Block size          | k = 1024 | k = 1536       | k = 2048 |
| n = 256             | 1061.18  | 1134.82        | 1162.93  |
| n = 288             | 1084.33  | 1153.30        | 1163.73  |
| n = 480             | 1164.31  | <b>1221.92</b> | 920.16   |
| n = 512             | 1167.48  | 1071.58        | 896.62   |
| n = 576             | 1166.31  | 1070.00        | 859.40   |

**Table 8: Experiment 2: Performance of Tiling\_A with Four Threads (GFLOPS)**

| Tiling_A four threads |          |          |                |
|-----------------------|----------|----------|----------------|
| Block size            | k = 1024 | k = 1536 | k = 2048       |
| n = 256               | 4026.46  | 4342.00  | 4424.89        |
| n = 288               | 4122.29  | 4409.90  | <b>4452.13</b> |
| n = 480               | 4375.38  | 4393.10  | 3564.22        |
| n = 512               | 4411.32  | 4235.34  | 3416.13        |
| n = 576               | 4438.77  | 4020.40  | 3317.26        |

**Table 9: Experiment 2: Performance of Tiling\_A with Eight Threads (GFLOPS)**

| Tiling_A eight threads |          |                |          |
|------------------------|----------|----------------|----------|
| Block size             | k = 1024 | k = 1536       | k = 2048 |
| n = 256                | 7502.52  | 8172.69        | 7997.11  |
| n = 288                | 7577.21  | 8228.45        | 8154.56  |
| n = 480                | 7958.21  | 8341.36        | 6931.99  |
| n = 512                | 8094.64  | <b>8441.28</b> | 6591.91  |
| n = 576                | 8115.99  | 7599.61        | 6405.60  |

**Table 10: Experiment 2: Performance of Tiling\_B with One Thread (GFLOPS)**

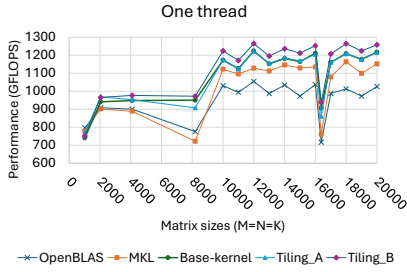
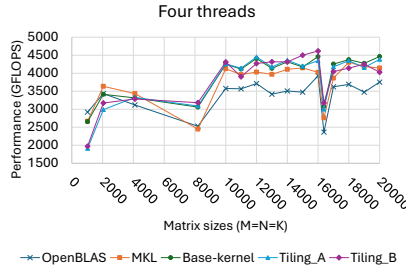
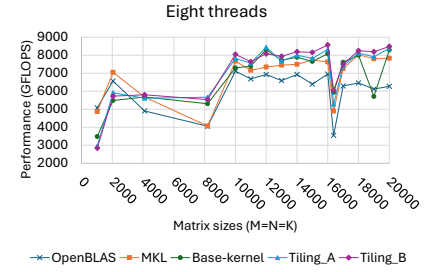
| Tiling_B one thread |          |                |          |
|---------------------|----------|----------------|----------|
| Block size          | k = 1024 | k = 1536       | k = 2048 |
| n = 256             | 1065.43  | 1135.23        | 1160.57  |
| n = 288             | 1127.96  | 1194.78        | 1220.40  |
| n = 480             | 1205.64  | <b>1264.62</b> | 870.45   |
| n = 512             | 1211.71  | 1049.83        | 821.27   |
| n = 576             | 1221.33  | 1165.47        | 736.36   |

**Table 11: Experiment 2: Performance of Tiling\_B with Four Threads (GFLOPS)**

| Tiling_B four threads |          |                |          |
|-----------------------|----------|----------------|----------|
| Block size            | k = 1024 | k = 1536       | k = 2048 |
| n = 256               | 4046.50  | 4346.22        | 4437.41  |
| n = 288               | 4267.60  | 4582.61        | 4560.77  |
| n = 480               | 4533.81  | <b>4617.80</b> | 3382.62  |
| n = 512               | 4574.37  | 4184.15        | 3059.65  |
| n = 576               | 4607.08  | 3984.95        | 2865.60  |

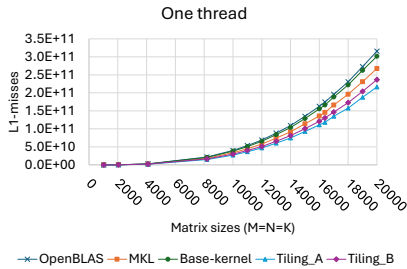
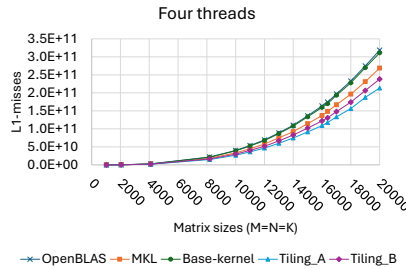
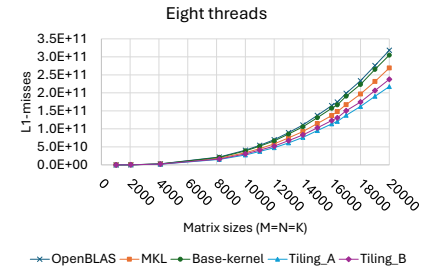
**Table 12: Experiment 2: Performance of Tiling\_B with Eight Threads (GFLOPS)**

| Tiling_B eight threads |          |                |          |
|------------------------|----------|----------------|----------|
| Block size             | k = 1024 | k = 1536       | k = 2048 |
| n = 256                | 7488.87  | 8177.76        | 8074.41  |
| n = 288                | 7824.84  | 8507.55        | 8381.71  |
| n = 480                | 8156.42  | <b>8568.43</b> | 6570.15  |
| n = 512                | 8318.49  | 8522.25        | 6081.47  |
| n = 576                | 8353.06  | 7491.11        | 5465.48  |

**Figure 14: Experiment 2: Best performance comparison with one thread.****Figure 15: Experiment 2: Best performance comparison with four threads.****Figure 16: Experiment 2: Best performance comparison with eight threads.**

The theoretical peak performance of AMX was calculated based on its operations per cycle and the base clock frequency of the processor because the actual clock frequency during AMX execution is not published ( $1024 \text{ Operations/cycle} \times 1.9 \text{ GHz base clock} = 1945.6 \text{ GFLOPS}$ ). Since this calculation is based on the base clock frequency, the actual peak performance could be slightly higher.

In single-threaded execution, our best implementation (Tiling\_B) shows an improvement of over 9% in efficiency compared to that of the original OpenBLAS and over 5% compared to that of MKL.

**Figure 17: Number of L1 cache misses for each implementation with one thread.****Figure 18: Number of L1 cache misses for each implementation with four threads.****Figure 19: Number of L1 cache misses for each implementation with eight threads.**

**Table 13: Efficiency Relative to Theoretical Peak Performance (%)**

|             | One Thread | Four Threads | Eight Threads |
|-------------|------------|--------------|---------------|
| OpenBLAS    | 54.2       | 50.5         | 45.7          |
| MKL         | 59.8       | 55.3         | 51.2          |
| Base-kernel | 62.9       | 56.4         | 53.4          |
| Tiling_A    | 62.8       | 57.2         | 54.2          |
| Tiling_B    | 65.0       | 59.3         | 55.1          |

## 7 Conclusion and Future Work

### 7.1 Conclusion

In this research, we focused on accelerating BFloat16 GEMM by leveraging the Intel AMX instruction set. We proposed and implemented novel tile operations to optimize the data layout and adjusted cache blocking sizes to improve efficiency. Through performance evaluations against existing libraries, we demonstrated that our proposed kernel surpasses both MKL and OpenBLAS, particularly for large-scale matrices. The implemented optimizations achieved performance improvements ranging from 7.27% to a maximum of 20.40%. These results confirmed that our proposed combination of tile operations and block sizes effectively harnessed the full computational potential of AMX. The results of this research contribute to the high performance of simulations and applications utilizing matrix multiplication.

### 7.2 Future Work

As extensions of this research, we identify the following future challenges:

- **Performance evaluation on multi-NUMA/socket systems:** Evaluating the performance on systems with two or more NUMA nodes or sockets to analyze scalability and identify potential bottlenecks in larger-scale hardware environments.
- **Implementation of GEMM with guaranteed accuracy:** Extending the current work to implement GEMM with guaranteed accuracy, which would use AMX to achieve pseudo single-precision (or higher) accuracy from BFloat16 computations.
- **Support for batched GEMM:** Implementing and optimizing batched GEMM to process multiple small matrices simultaneously. This functionality is in high demand for various applications, particularly in deep learning inference.

## Acknowledgments

This work was partially supported by JSPS KAKENHI Grant Number 24K02949. The computation was carried out using the computer resource offered under the category of General Projects by Research Institute for Information Technology, Kyushu University.

## References

- [1] Ahmad Abdelfattah, Stanimire Tomov, and Jack Dongarra. 2019. *Optimizing Batched HGEEM on Small Sizes Using Tensor Cores*. Technical Report ICL-UTK-1234-2019. Innovative Computing Laboratory, University of Tennessee. <https://icl.utk.edu/files/publications/2019/icl-utk-1234-2019.pdf> Also presented as a poster at the GPU Technology Conference (GTC) 2019, San Jose, CA.
- [2] Intel. 2025. *Intel® Intrinsics Guide*. <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>
- [3] Intel. 2025. *What Is Intel® Advanced Matrix Extensions?* <https://www.intel.com/content/www/us/en/products/docs/accelerator-engines/what-is-intel-amx.html> Accessed: 2025-07-02.
- [4] Norman P. Jouppi, Cliff Young, Nishant Patil, et al. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*. 1–12.
- [5] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, Jiyan Yang, Jongsoo Park, Alexander Heinecke, Evangelos Georganas, Sudarshan Srinivasan, Abhisek Kundu, Misha Smelyanskiy, Bharat Kaul, and Pradeep Dubey. 2019. A Study of BFloat16 for Deep Learning Training. *arXiv:1905.12322* [cs.LG]
- [6] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. 2018. NVIDIA Tensor Core Programmability, Performance & Precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*.
- [7] Matt Martineau, Patrick Atkinson, and Simon McIntosh-Smith. 2018. Benchmarking the NVIDIA V100 GPU and Tensor Cores. In *Euro-Par 2018: Parallel Processing Workshops*. 444–455.
- [8] Daichi Mukunoki, Katsuhisa Ozaki, Takeshi Ogita, and Toshiyuki Imamura. 2020. DGEMM Using Tensor Cores, and Its Accurate and Reproducible Versions. *Scientific Programming* 2020 (2020), 8821958:1–8821958:14. <https://doi.org/10.1155/2020/8821958>
- [9] Hiroyuki Ootomo, Katsuhisa Ozaki, and Rio Yokota. 2024. DGEMM on Integer Matrix Multiplication Unit. *International Journal of High Performance Computing Applications* 38, 4 (2024), 297–313. <https://doi.org/10.1177/10943420241239588>
- [10] OpenBLAS Project. 2025. *OpenBLAS*. <https://www.openblas.net/>
- [11] perf community. 2025. *perf: Linux profiling with performance counters*. <https://perf.wiki.github.io/main/>
- [12] Paul Springer and Paolo Bientinesi. 2017. Design of a High-Performance GEMM-like Tensor-Tensor Multiplication. *ACM Transactions on Mathematical Software (TOMS)* 44, 3 (2017), 28:1–28:29.
- [13] Hao Tang, Kazuhiko Komatsu, Masayuki Sato, and Hiroaki Kobayashi. 2021. Efficient Mixed-Precision Tall-and-Skinny Matrix-Matrix Multiplication for GPUs. *International Journal of Networking and Computing* 11, 2 (2021), 267–282.
- [14] Pedro Valero-Lara, Ian Jorquera, Frank Lui, and Jeffrey S. Vetter. 2023. Mixed-Precision S/DGEMM Using the TF32 and TF64 Frameworks on Low-Precision AI Tensor Cores. In *Proceedings of the 7th International Workshop on Software Correctness for HPC Applications (Correctness '23)*. Association for Computing Machinery, New York, NY, USA. [https://sc23.supercomputing.org/proceedings/workshops/workshop\\_pages/ws\\_corr101.html](https://sc23.supercomputing.org/proceedings/workshops/workshop_pages/ws_corr101.html)
- [15] Da Yan, Wei Wang, and Xiaowen Chu. 2020. Demystifying Tensor Cores to Optimize Half-Precision Matrix Multiply. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 215–226. <https://doi.org/10.1109/IPDPS47924.2020.00030>
- [16] Shaoshuai Zhang, Elaheh Baharlouei, and Panrui Wu. 2020. High Accuracy Matrix Computations on Neural Engines: A Study of QR Factorization and its Applications. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '20)*. Association for Computing Machinery, New York, NY, USA, 17–28. <https://doi.org/10.1145/3369583.3392680>